

Lesson 22 — Parameters — One Function, Many Behaviors

Matches LEGO Teacher Portal lesson: "Funky Functions".

Objective (I can...): I can give a function a parameter so one definition handles many cases. **CS concept:** Parameters and arguments · **Builds on:** Lesson 21 (define/call).

AI Connection (2 min): A reusable "classify(image)" takes the image as a parameter — same function, different input each time. *First-principles question:* "Why is a function that takes an input more powerful than one hard-wired to a single value?"

Materials: LEGO CS & AI kit + device per pair, notebooks. **Model reuse:** Functions model from Lesson 21 — code only.

60-minute flow - Warm-up (8): "Same recipe, different amount" — coach gives a routine that only differs by a number ("drive 10," "drive 25"); pairs spot that the one thing that changes should be the parameter. - **Concept + demo (10):** Define a function with one parameter (e.g., "driveFor(distance)" or "flash(times)"); call it with different arguments and show different behavior from one definition. (*Rotate format: "trace it on paper first" — predict each call's result.*) - **Build/Code (30):** Pairs convert a fixed-value function from last lesson into a parameterized one, then call it with at least three different arguments. They log, in the notebook, what the parameter controls. *Swap at 15 min.* - **Challenge (7):** Add a second parameter so one call controls two things (distance and speed). - **Share + pack-up (5):** Pairs demo three calls with different arguments and explain what the parameter changed. Model stays built.

Pair roles: Driver codes the parameter, Navigator chooses and logs the test arguments. *Swap at 15 min. (Trio: third runs each argument.)*

Differentiation: *Support* — provide the parameterized definition; students supply varied arguments. *Extension / fast-finisher* — two parameters, or a parameter used inside a loop count. *Reteach* — change the hard-coded number by hand a few times, then replace it with a parameter together.

Exit check: Pair's parameterized function produces different, correct behavior for different arguments.

At-home (optional): Write a real routine as a function with a parameter (e.g., "cook(minutes)"); show three calls. ~10 min.